Predictions:

E1: How fast? Run Protocol A on T1–T4. Plot δ[k] on a log scale, all on one figure, and report kε for each. Repeat T4 with r = rmin + 0.4. Rank the networks and explain the ranking using

```
    %your Part I table. What does the gap between the two zones do?

% We would expect the topologies with the higher edge counts to rendezvous
% quicker as they have more connections and therefore more information.
% For that reason we expect Complete to
% perform the greatest, Disk with added radius and Star to perform somewhat
worse,
% and Ring/Disk with rmin to perform the worst.

% Protocol A convergence comparison
rng(105);
P = [5*rand(50,2); 5*rand(50,2) + [6 0]];
n = size(P,1);

%Make adjacency matrices
G_complete = make_complete(n);
G_star = make_star(n);
G_ring = make_ring(n, P);
G_disk = make_disk(P);
G_large_disk = make_disk_offset(P);

%Make movement matrices
W_complete = Establish_Matrices(G_complete, 1);
W_star = Establish_Matrices(G_star, 1);
W_ring = Establish_Matrices(G_ring, 1);
W_disk = Establish_Matrices(G_disk, 1);
W_large_disk = Establish_Matrices(G_large_disk, 1);

%Obtain rendezvous times
[d_complete,k_complete] = Spread(W_complete, P);
[d_star,k_star] = Spread(W_star, P);
[d_ring,k_ring] = Spread(W_ring, P);
[d_disk,k_disk] = Spread(W_disk, P);
[d_large_disk,k_large_disk] = Spread(W_large_disk, P);

%Find longest rendezvous time for outer limit of the graph
N = max([length(d_complete), length(d_star), length(d_ring), length(d_disk),
length(d_large_disk)]);

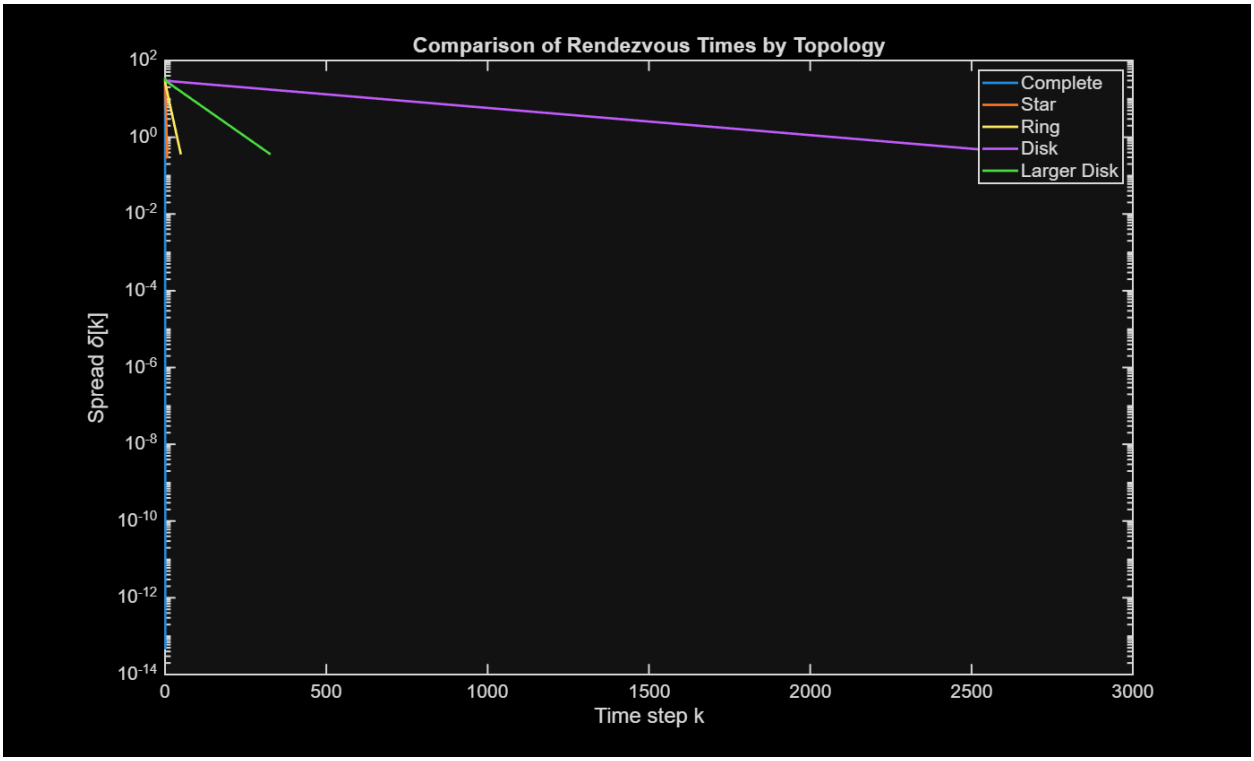
steps = 0:N-1; %x-axis for steps
```

```
spreads = NaN(N,5); %we can't initialize the y-axis as 0s by default since
we're graphing them

spreads(1:length(d_complete),1) = d_complete(:);
spreads(1:length(d_star),2) = d_star(:);
spreads(1:length(d_ring),3) = d_ring(:);
spreads(1:length(d_disk),4) = d_disk(:);
spreads(1:length(d_large_disk),5) = d_large_disk(:);

fig = figure;
ax = axes(fig);
plot(ax, steps, spreads, 'LineWidth', 2);

set(ax, 'FontSize', 16, 'LineWidth', 1.5);
yscale(ax, 'log');
xlabel(ax, 'Time step k', 'FontSize', 18);
ylabel(ax, 'Spread \delta[k]', 'FontSize', 18);
legend(ax, {'Complete', 'Star', 'Ring', 'Disk', 'Larger Disk'}, ...
    'FontSize', 16);
title('Comparison of Rendezvous Times by Topology')
```



Experiment 1 Explanation

```
% 1) Complete is fastest, as all 100 robots immediately average to the same
% position and rendezvous in one step. This is demonstrated in the data
% table as its diameter of 1 means that each node is connected to every
% other node.
% 2) Star is the next fastest, as although it has much less edges than
```

```
% Complete, its diameter of 2 means that the central node connects each
% other robot indirectly. A lot of averaging will therefore occur during
% each step.
% 3) Ring is the next best, despite having a similar edge count and
% diameter to disk, so the structural difference between the two is not
% apparent in the data table. However, ring will group nodes based on
% indices and not physical position, so far away nodes will still be
% brought together over time.
% 4) The larger disk performs substantially worse, as neighboring nodes are
% decided based on their physical position. This will lead to cases where
% two distant clusters of nodes spend many averaging steps getting closer
% together, and hundreds more bridging the gap between each other.
% 5) Regular disk performs the worst, as it has less connections with a
% lower radius and will struggle to bridge distant groups of nodes during
% averaging steps.

% The gap between the two zones creates a bottleneck for each topology that
% almost solely determines how quickly they can rendezvous, where more
% links bridging those zones correlates to faster rendezvous time. Since
% complete, star, and ring do not rely on physical distance, they perform
% the best.

% Our prediction was right that Complete would be the best performer and
% Disk the worst, but every other prediction inbetween was wrong and our
% methodology did not hold up either. Edge count did not show an actual
% bearing on rendezvous time, as disk and ring with similar edge counts had
% wildly different outcomes. We did not anticipate the challenge that disk
% would face when forming two clusters of nodes and how significantly that
% bottleneck would affect speed.
```

E2: Where do they meet? Using part (c) of the Checkpoint, predict the rendezvous point

for each protocol as a formula in x[0] and y[0]. Verify your formulas numerically on every

network. On the star, how far from the swarm's average position does each protocol meet,

and why? Which protocol reaches rendezvous faster on the star, and why?

```
x0 = P(:,1);
y0 = P(:,2);
average_position = mean(P,1);
```

```
K = 3000; % Disk took 2700~ time steps to converge, so we give it some
wiggle room

for network = 1:4
    switch network
        case 1
            G = G_complete;
            name = 'Complete';
        case 2
            G = G_star;
            name = 'Star';
        case 3
            G = G_ring;
            name = 'Ring';
        case 4
            G = G_disk;
            name = 'Disk';
    end

    d = G*ones(n,1);
    weights = 1 + d;

    % Each prediction is a row vector
    predicted_A = (weights'*P)/sum(weights);
    predicted_B = average_position;

    fprintf('\n%s network\n',name);

    for protocol = 1:2
        W = Establish_Matrices(G,protocol);

        X = simulate_linear(W,x0,K);
        Y = simulate_linear(W,y0,K);

        if protocol == 1
            predicted = predicted_A;
        else
            predicted = predicted_B;
        end

        % Distance of every final robot position from our prediction
        errors = sqrt( ...
            (X(:,end)-predicted(1)).^2 + ...
            (Y(:,end)-predicted(2)).^2);

        fprintf('Protocol %d predicted point: (%.6f, %.6f)\n', ...
            protocol,predicted(1),predicted(2));

        fprintf('Maximum final distance from prediction: %.3e\n', ...
            max(errors));

        if network == 2
            offset = norm(predicted-average_position);
            [~,rendezvous_time] = Spread(W,P);
```

```
            fprintf('Distance from initial swarm average: %.6f\n', ...
                offset);
            fprintf('Rendezvous time: %g steps\n', rendezvous_time);
        end
    end
end


% Rendezvous point of protocol A is the summation of (1+d_i)x0/
summation(1+d_i); this means that the rendezvous point is slightly off-
centered as
% drones with larger degrees are weighted more, causing it to move less
% then the others to the actual rendezvous point, and the same pattern is
found on the y-axis.

% The rendezvous point of protocol B is the average of all the drones,
% given by the fact that the end formula results in 1^Tx0/n, and the same on
the y-axis.

% For protocol A, the rendezvous point ends up being approximately 1/3
% of the way between the location of the central drone and the swarms
average.
% This is because the central drone is an extreme case where one drone has
all of the weight, causing it to move very little relative to the rest of the
% swarm towards the average position, slowly altering it as drones approach.
%

% For protocol B, the robots converge on the exact average point as given by
the above equation for the rendezvous point for protocol B

% Protocol A will reach the rendezvous point faster for star as it will
immediately jump to the average + bias, rather than B where they take small
% steps towards the rendezvous point. And because as the number of drones
get larger, the smaller each step, as the number of drones increases,
% the required steps also increase.

% Our predictions were therefore generally correct.


function G = make_disk_offset(P)
%Since make_disk determines rmin at runtime, we restate the function with
%an override here that sets r 0.4 higher

    addpath("part1\")
    %init
    n = size(P, 1);
    s = sum(P.^2, 2); %square each element of P, then add each x and y
    one = ones(n, 1);
    r = 0.1;
    while true

        delta = s*one' + one*s' - 2*(P*P'); %effectively creating a star
topology while encoding distances
```

```
        G = delta <= r^2; %only values within range are 1 and rest 0
        G(1:n+1:end) = 0; %set diagonals to 0

        stats = graph_stats(G);
        diameter = stats(4);

        if ~isinf(diameter) %if no isolated nodes
            r = r + 0.4; %rmin + 0.4, then recalculate adjacency
            delta = s*one' + one*s' - 2*(P*P');
            G = delta <= r^2;
            G(1:n+1:end) = 0;
            break;
        end
        r = r + 0.1; %try again with higher radius

    end

end
```

```
Complete network
Protocol 1 predicted point: (5.366138, 2.649444)
Maximum final distance from prediction: 1.344e-14
Protocol 2 predicted point: (5.366138, 2.649444)
Maximum final distance from prediction: 1.257e-14

Star network
Protocol 1 predicted point: (3.739128, 3.337753)
Maximum final distance from prediction: 1.708e-14
Distance from initial swarm average: 1.766616
Rendezvous time: 7 steps
Protocol 2 predicted point: (5.366138, 2.649444)
Maximum final distance from prediction: 1.192e-12
Distance from initial swarm average: 0.000000
Rendezvous time: 453 steps

Ring network
Protocol 1 predicted point: (5.366138, 2.649444)
Maximum final distance from prediction: 2.306e-14
Protocol 2 predicted point: (5.366138, 2.649444)
Maximum final distance from prediction: 6.764e-14

Disk network
Protocol 1 predicted point: (5.456527, 2.679148)
Maximum final distance from prediction: 2.311e-02
Protocol 2 predicted point: (5.366138, 2.649444)
Maximum final distance from prediction: 1.432e-01
```

Published with MATLAB® R2026a